

Jeu 1 : Attrape

Des objets tombent du ciel. Le bon objet donne 10 points, le mauvais enlève une vie (3 vies). Le joueur se déplace en bas avec les flèches gauche et droite. Le jeu a trois écrans : menu, partie, fin. Le joueur est une seule image.

Ce guide se suit dans l'ordre. Après chaque étape il y a un test : si tu ne vois pas ce qui est annoncé, ne passe pas à la suite.

Le code utilise seulement ce qui a été vu en cours :

- le CM et le TD1 pour le joueur, les groupes, les collisions et le score ;
- le TD3 pour les fichiers, les scènes, `window.score` et les sons.

Étape 0 : préparer le thème et les fichiers

Les images du thème

Pour un nouveau thème (dauphin, Noël, rocher, voiture, lunettes...), seules ces images changent. Garde le même nom de fichier.

Fichier	Taille	À quoi ça sert	Idée
<code>fond.png</code>	au moins 800 x 600	fond de tous les écrans	décor lié au mot
<code>sol.png</code>	exactement 400 x 32	bande en bas, affichée en double largeur	herbe, sable, glace, route
<code>joueur.png</code>	environ 32 x 48	le personnage, une seule image	personnage ou objet du thème, tourné vers la droite
<code>bon.png</code>	environ 40 x 40	rapporte 10 points	un élément du thème
<code>mauvais.png</code>	environ 30 x 30	enlève une vie	un intrus du thème

Pour les trouver :

1. Recherche sur internet « mot du thème png transparent » (le fond transparent est important pour `bon`, `mauvais` et `joueur`).
2. Enregistre l'image au format PNG.
3. Redimensionne-la à la taille du tableau avec un site de redimensionnement d'image ou avec Paint (Redimensionner, en pixels, sans conserver les proportions si besoin).
4. Renomme-la avec le nom du tableau.

Le joueur : une seule image

Le joueur est une seule image, sans animation. Trois conseils :

- Taille : environ 32 x 48 pixels, de la même taille que celle prévue dans le tableau. Phaser utilise la taille de l'image comme zone de collision : une image trop grande donne un joueur énorme.

- Orientation : l'image doit regarder vers la droite. Quand le joueur va à gauche, le code retourne l'image avec `flipX` (TD3).
- Fond transparent, comme pour les autres images.

Les fichiers à récupérer sur le réseau

- `phaser.min.js` : dans le dossier `js` du TD3. Prends bien celui-là (version 3.55), les versions récentes gèrent différemment certaines choses.
- Les sons du TD1 : `ding.mp3` devient `bon.mp3`, `criMort.mp3` devient `mauvais.mp3`.

Les dossiers

Dans ton W, crée cette arborescence. Elle est identique à celle du TD3.



Crée pour l'instant les dossiers, mets-y les images, les sons et `phaser.min.js`, et crée des fichiers vides pour tous les autres.

Lancer le jeu

Ouvre toujours le jeu avec l'adresse du serveur web de l'IUT, pas en double-cliquant sur `index.html`. Sinon la page reste vide (le navigateur bloque les modules et le chargement des images). Après chaque modification, enregistre le fichier et recharge la page (F5 ou Ctrl + F5).

Pour voir les erreurs : touche F12, onglet Console.

Étape 1 : `index.html`

Il charge le style, Phaser, puis `main.js`. L'ordre compte : Phaser avant `main.js`. `type="module"` permet à `main.js` d'importer les scènes.

```

<!doctype html>
<html lang="fr">
<head>
  <meta charset="UTF-8" />
  <title>Attrape</title>
  <link rel="stylesheet" href="dist/css/main.css">
  <script src="dist/js/lib/phaser.min.js"></script>
  <script src="dist/js/main.js" type="module"></script>
</head>
<body>
</body>
</html>

```

Étape 2 : dist/css/main.css

```

body {
  margin: 0;
  background: #2d2d2d;
}

```

Étape 3 : les trois scènes vides

Chaque scène suit le modèle du TD3. Pour les trois fichiers, change seulement le nom (trois endroits : `class`, `super`, `export`).

dist/js/scenes/menu.js :

```

class Menu extends Phaser.Scene {
  constructor() {
    super('Menu');
  }

  preload() {
  }

  create() {
  }

  update(time, delta) {
  }
}

export default Menu;

```

Fais pareil pour `game.js` (classe `Game`, `super('Game')`) et `end.js` (classe `End`, `super('End')`). Garde la majuscule.

Étape 4 : dist/js/main.js

Ce fichier importe les scènes et crée le jeu. La première scène de la liste `scene` est celle qui démarre : le menu.

```

import Menu from "../scenes/menu.js";
import Game from "../scenes/game.js";
import End from "../scenes/end.js";

// variable globale : on la retrouve dans toutes les scènes
window.score = 0;

window.config = {
  type: Phaser.AUTO,
  width: 800,
  height: 600,
  scale: {
    mode: Phaser.Scale.FIT,
    autoCenter: Phaser.Scale.CENTER_BOTH
  },
  backgroundColor: '#2d2d2d',
  scene: [Menu, Game, End],
  physics: {
    default: 'arcade',
    arcade: {
      gravity: { y: 300 },
      debug: false
    }
  }
};

const game = new Phaser.Game(window.config);

```

Test. La page affiche un rectangle gris foncé centré. La console ne montre aucune erreur rouge.

Si la page est vide, vérifie : l'adresse (serveur, pas fichier), les chemins dans `index.html`, et que les fichiers de scène existent avec `export default`.

Étape 5 : le menu

Dans `menu.js`, on charge le fond, puis on affiche le titre, les règles et un bouton. Le bouton lance la scène `Game` (TD3, étape 6).

Dans `preload()` :

```

this.load.image('fond', 'dist/assets/img/fond.png');

```

Dans `create()` :

```

    this.add.image(400, 300, 'fond');

    this.add.text(400, 160, 'Attrape', { fontFamily: 'Georgia', fontSize: '80px', fill:
'#000' })
        .setOrigin(0.5);

    this.add.text(400, 300,
        'Flèches gauche et droite pour bouger.\n' +
        'Le bon objet rapporte 10 points.\n' +
        'Le mauvais te coûte une vie. Tu en as 3.',
        { fontFamily: 'Courier New', fontSize: '22px', fill: '#000', align: 'center' })
        .setOrigin(0.5);

    this.add.text(400, 450, 'Jouer', { fontFamily: 'Georgia', fontSize: '48px', fill:
'#fff', backgroundColor: '#000' })
        .setOrigin(0.5)
        .setInteractive()
        .on('pointerdown', () => {
            this.scene.start('Game');
        });

```

`setOrigin(0.5)` centre le texte sur le point donné. Change le titre et les phrases pour qu'ils collent à ton thème.

Test. Le fond et le menu s'affichent. Un clic sur « Jouer » donne un écran gris (la scène `Game` est vide pour l'instant).

Étape 6 : la scène de jeu, fond et sol

Dans `game.js`, remplis `preload()` puis `create()`.

```

preload() {
    this.load.image('fond', 'dist/assets/img/fond.png');
    this.load.image('sol', 'dist/assets/img/sol.png');
}

```

```

create() {
    this.add.image(400, 300, 'fond');

    // le sol
    this.platforms = this.physics.add.staticGroup();
    this.platforms.create(400, 568, 'sol').setScale(2).refreshBody();
}

```

Le sol est un groupe statique : il ne bouge pas et la gravité ne l'affecte pas (CM, étape 3). `setScale(2)` double sa largeur pour couvrir les 800 pixels, `refreshBody()` met à jour sa zone de collision.

Test. Le jeu affiche le fond et le sol en bas.

Étape 7 : le joueur

Dans `preload()`, ajoute le chargement de l'image du joueur :

```
this.load.image('joueur', 'dist/assets/img/joueur.png');
```

Dans `create()`, à la suite du sol :

```
// le joueur
this.player = this.physics.add.sprite(400, 450, 'joueur');
this.player.setBounce(0.2);
this.player.setCollideWorldBounds(true);
this.physics.add.collider(this.player, this.platforms);

this.cursors = this.input.keyboard.createCursorKeys();
```

Le joueur est un sprite à physique dynamique : la gravité s'applique, il rebondit un peu (`setBounce`) et ne sort pas de l'écran (`setCollideWorldBounds`). Le `collider` l'empêche de traverser le sol.

Dans `update(time, delta)`, qui tourne environ 60 fois par seconde (CM, étape 5) :

```
// déplacement du joueur
if (this.cursors.left.isDown) {
    this.player.setVelocityX(-300);
    this.player.flipX = true;
} else if (this.cursors.right.isDown) {
    this.player.setVelocityX(300);
    this.player.flipX = false;
} else {
    this.player.setVelocityX(0);
}
```

`flipX = true` retourne l'image pour regarder à gauche, `false` la remet à droite.

Test. Le joueur tombe sur le sol et se déplace à gauche et à droite. Il se retourne selon la direction et ne sort pas de l'écran.

Étape 8 : les objets qui tombent

Il y a deux groupes dynamiques : les bons objets et les mauvais (TD1, partie 3 : « Créer un groupe dynamique »).

Dans `preload()`, ajoute :

```
this.load.image('bon', 'dist/assets/img/bon.png');
this.load.image('mauvais', 'dist/assets/img/mauvais.png');
```

Tout en haut de `create()`, avant le fond, ajoute les valeurs de départ. `create` est rappelé à chaque nouvelle partie, donc c'est ici qu'on remet tout à zéro.

```
// valeurs de départ (create est rappelé à chaque nouvelle partie)
window.score = 0;
this.vies = 3;
this.delai = 0; // temps écoulé depuis le dernier objet
this.intervalle = 800; // temps entre deux objets (ms)
```

À la fin de `create()`, ajoute les groupes :

```
// les deux types d'objets qui tombent
this.bons = this.physics.add.group();
this.mauvais = this.physics.add.group();
```

À la fin de `update()`, on fait apparaître un objet de temps en temps. C'est la même idée que les nuages du TD3 : on utilise `delta` (le temps entre deux images) pour compter.

```
// on fait tomber un nouvel objet de temps en temps
this.delai += delta;
if (this.delai > this.intervalle) {
    this.delai = 0;
    let x = Math.random() * 760 + 20;

    // 7 fois sur 10 c'est un bon objet
    if (Math.random() < 0.7) {
        this.bons.create(x, 0, 'bon');
    } else {
        this.mauvais.create(x, 0, 'mauvais');
    }
}
```

`Math.random()` donne un nombre entre 0 et 1. `* 760 + 20` en fait une position entre 20 et 780.

Test. Des objets tombent d'en haut toutes les 0,8 seconde environ, de deux types. Ils traversent le sol (pas encore de collision) et le joueur.

Étape 9 : collisions, score et vies

À la fin de `create()`, ajoute les collisions et les textes. Ordre : après la création des groupes.

```
// un objet qui touche le sol disparaît
this.physics.add.collider(this.bons, this.platforms, this.toucheSol, null, this);
this.physics.add.collider(this.mauvais, this.platforms, this.toucheSol, null, this);

// collisions avec le joueur
this.physics.add.overlap(this.player, this.bons, this.attraperBon, null, this);
this.physics.add.overlap(this.player, this.mauvais, this.attraperMauvais, null, this);

// textes
this.texteScore = this.add.text(16, 16, 'Score : 0', { fontFamily: 'Courier New',
fontSize: '28px', fill: '#000' });
this.texteVies = this.add.text(784, 16, 'Vies : 3', { fontFamily: 'Courier New',
fontSize: '28px', fill: '#000' })
.setOrigin(1, 0);
```

Le `this` en dernier argument dit à Phaser que les fonctions de rappel sont celles de la scène.

Après `update()`, dans la classe `Game`, ajoute les trois fonctions appelées par les collisions :

```

toucheSol(objet, sol) {
    objet.disableBody(true, true);
}

attraperBon(player, bon) {
    bon.disableBody(true, true);

    window.score += 10;
    this.texteScore.setText('Score : ' + window.score);
}

attraperMauvais(player, mauvais) {
    mauvais.disableBody(true, true);

    this.vies--;
    this.texteVies.setText('Vies : ' + this.vies);

    if (this.vies === 0) {
        this.scene.start('End');
    }
}

```

`disableBody(true, true)` fait disparaître l'objet (CM, étape 6).

Test. Attraper un bon objet fait monter le score de 10. Un mauvais fait baisser les vies. À 0 vie l'écran devient gris (la scène `End` est vide). Les objets ratés disparaissent au sol.

Étape 10 : les sons

Dans `preload()` :

```

this.load.audio('sonBon', 'dist/assets/sounds/bon.mp3');
this.load.audio('sonMauvais', 'dist/assets/sounds/mauvais.mp3');

```

À la fin de `create()` :

```

// sons
this.sonBon = this.sound.add('sonBon');
this.sonMauvais = this.sound.add('sonMauvais');

```

Dans `attraperBon`, juste après `bon.disableBody(true, true);` :

```

this.sonBon.play();

```

Dans `attraperMauvais`, juste après `mauvais.disableBody(true, true);` :

```

this.sonMauvais.play();

```

Test. Un son à chaque objet attrapé. Si tu n'entends rien, clique d'abord dans la page : certains navigateurs bloquent le son avant le premier clic.

Étape 11 : le jeu s'accélère

Dans `attraperBon`, à la fin :

```
// le jeu s'accélère petit à petit
if (this.intervalle > 300) {
  this.intervalle -= 15;
}
```

Chaque bon objet attrapé rapproche les chutes de 15 ms, jusqu'à un minimum de 300 ms.

Étape 12 : l'écran de fin

Dans `end.js`. Le score vient de la scène `Game` par la variable globale `window.score`.

Dans `preload()` :

```
this.load.image('fond', 'dist/assets/img/fond.png');
```

Dans `create()` :

```
this.add.image(400, 300, 'fond');

this.add.text(400, 170, 'Perdu', { fontFamily: 'Georgia', fontSize: '80px', fill:
'#000' })
  .setOrigin(0.5);

// window.score vient de la scène Game
this.add.text(400, 290, 'Score : ' + window.score, { fontFamily: 'Courier New',
fontSize: '36px', fill: '#000' })
  .setOrigin(0.5);

this.add.text(400, 400, 'Rejouer', { fontFamily: 'Georgia', fontSize: '48px', fill:
'#fff', backgroundColor: '#000' })
  .setOrigin(0.5)
  .setInteractive()
  .on('pointerdown', () => {
    this.scene.start('Game');
  });

this.add.text(400, 480, 'Retour au menu', { fontFamily: 'Georgia', fontSize: '28px',
fill: '#fff', backgroundColor: '#000' })
  .setOrigin(0.5)
  .setInteractive()
  .on('pointerdown', () => {
    this.scene.start('Menu');
  });
```

Test final. Menu, partie, fin, rejouer : tout s'enchaîne. Le score de la fin correspond à celui de la partie.

Problèmes fréquents

Ce qu'on voit	Cause probable
Page vide	jeu ouvert par le fichier et pas par le serveur ; erreur dans la console (F12)
<code>Phaser is not defined</code>	<code>phaser.min.js</code> absent ou chargé après <code>main.js</code>
Une image est remplacée par un carré avec une croix	mauvais nom ou mauvais chemin dans <code>preload</code> ; majuscules différentes
Le joueur traverse le sol	collider du joueur oublié, ou <code>refreshBody()</code> oublié
Rien ne tombe	<code>delta</code> oublié dans <code>update(time, delta)</code> ou groupes non créés
<code>this.toucheSol is not a function</code>	la fonction est en dehors de la classe <code>Game</code>
Le score ne bouge pas	<code>setText</code> oublié
Le jeu garde l'ancien score en rejouant	les valeurs de départ ne sont pas au début de <code>create()</code>

Pour aller plus loin

- Un son de musique en boucle (TD3).
- Le bouton plein écran (TD1).
- Une meilleure note enregistrée avec `window`.
- Des objets qui tombent à des vitesses différentes avec `setVelocityY`.

Annexe : code complet

Le code final de chaque fichier, pour comparer avec le tien.

`index.html`

```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="UTF-8" />
  <title>Attrape</title>
  <link rel="stylesheet" href="dist/css/main.css">
  <script src="dist/js/lib/phaser.min.js"></script>
  <script src="dist/js/main.js" type="module"></script>
</head>
<body>
</body>
</html>
```

dist/css/main.css

```
body {  
  margin: 0;  
  background: #2d2d2d;  
}
```

dist/js/main.js

```
import Menu from "../scenes/menu.js";  
import Game from "../scenes/game.js";  
import End from "../scenes/end.js";  
  
// variable globale : on la retrouve dans toutes les scènes  
window.score = 0;  
  
window.config = {  
  type: Phaser.AUTO,  
  width: 800,  
  height: 600,  
  scale: {  
    mode: Phaser.Scale.FIT,  
    autoCenter: Phaser.Scale.CENTER_BOTH  
  },  
  backgroundColor: '#2d2d2d',  
  scene: [Menu, Game, End],  
  physics: {  
    default: 'arcade',  
    arcade: {  
      gravity: { y: 300 },  
      debug: false  
    }  
  }  
};  
  
const game = new Phaser.Game(window.config);
```

```
class Menu extends Phaser.Scene {
  constructor() {
    super('Menu');
  }

  preload() {
    this.load.image('fond', 'dist/assets/img/fond.png');
  }

  create() {
    this.add.image(400, 300, 'fond');

    this.add.text(400, 160, 'Attrape', { fontFamily: 'Georgia', fontSize: '80px',
fill: '#000' })
      .setOrigin(0.5);

    this.add.text(400, 300,
      'Flèches gauche et droite pour bouger.\n' +
      'Le bon objet rapporte 10 points.\n' +
      'Le mauvais te coûte une vie. Tu en as 3.',
      { fontFamily: 'Courier New', fontSize: '22px', fill: '#000', align:
'center' })
      .setOrigin(0.5);

    this.add.text(400, 450, 'Jouer', { fontFamily: 'Georgia', fontSize: '48px',
fill: '#fff', backgroundColor: '#000' })
      .setOrigin(0.5)
      .setInteractive()
      .on('pointerdown', () => {
        this.scene.start('Game');
      });
  }

  update(time, delta) {
  }
}

export default Menu;
```

```

class Game extends Phaser.Scene {
  constructor() {
    super('Game');
  }

  preload() {
    this.load.image('fond', 'dist/assets/img/fond.png');
    this.load.image('sol', 'dist/assets/img/sol.png');
    this.load.image('bon', 'dist/assets/img/bon.png');
    this.load.image('mauvais', 'dist/assets/img/mauvais.png');
    this.load.image('joueur', 'dist/assets/img/joueur.png');

    this.load.audio('sonBon', 'dist/assets/sounds/bon.mp3');
    this.load.audio('sonMauvais', 'dist/assets/sounds/mauvais.mp3');
  }

  create() {
    // valeurs de départ (create est appelé à chaque nouvelle partie)
    window.score = 0;
    this.vies = 3;
    this.delai = 0; // temps écoulé depuis le dernier objet
    this.intervalle = 800; // temps entre deux objets (ms)

    this.add.image(400, 300, 'fond');

    // le sol
    this.platforms = this.physics.add.staticGroup();
    this.platforms.create(400, 568, 'sol').setScale(2).refreshBody();

    // le joueur
    this.player = this.physics.add.sprite(400, 450, 'joueur');
    this.player.setBounce(0.2);
    this.player.setCollideWorldBounds(true);
    this.physics.add.collider(this.player, this.platforms);

    this.cursors = this.input.keyboard.createCursorKeys();

    // les deux types d'objets qui tombent
    this.bons = this.physics.add.group();
    this.mauvais = this.physics.add.group();

    // un objet qui touche le sol disparaît
    this.physics.add.collider(this.bons, this.platforms, this.toucheSol, null,
this);

    this.physics.add.collider(this.mauvais, this.platforms, this.toucheSol, null,
this);

    // collisions avec le joueur
    this.physics.add.overlap(this.player, this.bons, this.attraperBon, null,
this);

    this.physics.add.overlap(this.player, this.mauvais, this.attraperMauvais,
null, this);

    // textes
    this.texteScore = this.add.text(16, 16, 'Score : 0', { fontFamily: 'Courier
New', fontSize: '28px', fill: '#000' });
    this.texteVies = this.add.text(784, 16, 'Vies : 3', { fontFamily: 'Courier

```

```

New', fontSize: '28px', fill: '#000' })
    .setOrigin(1, 0);

    // sons
    this.sonBon = this.sound.add('sonBon');
    this.sonMauvais = this.sound.add('sonMauvais');
}

update(time, delta) {
    // déplacement du joueur
    if (this.cursors.left.isDown) {
        this.player.setVelocityX(-300);
        this.player.flipX = true;
    } else if (this.cursors.right.isDown) {
        this.player.setVelocityX(300);
        this.player.flipX = false;
    } else {
        this.player.setVelocityX(0);
    }

    // on fait tomber un nouvel objet de temps en temps
    this.delai += delta;
    if (this.delai > this.intervalle) {
        this.delai = 0;
        let x = Math.random() * 760 + 20;

        // 7 fois sur 10 c'est un bon objet
        if (Math.random() < 0.7) {
            this.bons.create(x, 0, 'bon');
        } else {
            this.mauvais.create(x, 0, 'mauvais');
        }
    }
}

toucheSol(objet, sol) {
    objet.disableBody(true, true);
}

attraperBon(player, bon) {
    bon.disableBody(true, true);
    this.sonBon.play();

    window.score += 10;
    this.texteScore.setText('Score : ' + window.score);

    // le jeu s'accélère petit à petit
    if (this.intervalle > 300) {
        this.intervalle -= 15;
    }
}

attraperMauvais(player, mauvais) {
    mauvais.disableBody(true, true);
    this.sonMauvais.play();

    this.vies--;
    this.texteVies.setText('Vies : ' + this.vies);

    if (this.vies === 0) {

```

```

        this.scene.start('End');
    }
}

export default Game;

```

dist/js/scenes/end.js

```

class End extends Phaser.Scene {
    constructor() {
        super('End');
    }

    preload() {
        this.load.image('fond', 'dist/assets/img/fond.png');
    }

    create() {
        this.add.image(400, 300, 'fond');

        this.add.text(400, 170, 'Perdu', { fontFamily: 'Georgia', fontSize: '80px',
fill: '#000' })
            .setOrigin(0.5);

        // window.score vient de la scène Game
        this.add.text(400, 290, 'Score : ' + window.score, { fontFamily: 'Courier
New', fontSize: '36px', fill: '#000' })
            .setOrigin(0.5);

        this.add.text(400, 400, 'Rejouer', { fontFamily: 'Georgia', fontSize: '48px',
fill: '#fff', backgroundColor: '#000' })
            .setOrigin(0.5)
            .setInteractive()
            .on('pointerdown', () => {
                this.scene.start('Game');
            });

        this.add.text(400, 480, 'Retour au menu', { fontFamily: 'Georgia', fontSize:
'28px', fill: '#fff', backgroundColor: '#000' })
            .setOrigin(0.5)
            .setInteractive()
            .on('pointerdown', () => {
                this.scene.start('Menu');
            });
    }

    update(time, delta) {
    }
}

export default End;

```